

Progressive Development of a ROS 2 Ackermann Vehicle: Environment Validation, Simulation Driving Modes, and Closed-Loop Control

Andrew Abdelmalak

*Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt*

andrew.abdelmalak@student.guc.edu.eg

Daniel Boules

*Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt*

daniel.boules@student.guc.edu.eg

David Girgis

*Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt*

david.girgis@student.guc.edu.eg

Kirolous Kirolous

*Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt*

kirolous.kirolous@student.guc.edu.eg

Samir Yacoub

*Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt*

samir.yacoub@student.guc.edu.eg

Youssef Salama

*Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt*

youssef.salama@student.guc.edu.eg

Abstract—This paper documents the progressive development of Team 23’s autonomous Ackermann-steered vehicle through Milestones 1–5 of the MCTR1002 Autonomous Systems course. Milestone 1 established the ROS 2 Jazzy software foundation on two target platforms: a Raspberry Pi 4 hardware computer running Ubuntu Noble 24.04 and a desktop Gazebo Harmonic simulation environment. Milestone 2 extended this foundation into vehicle-level operation by launching an Ackermann model in an empty Gazebo world, bridging its command and state topics into ROS 2, implementing open-loop response (OLR) and keyboard teleoperation driving nodes, and testing a physical actuator stack through an Arduino-based motor and steering controller. Milestone 3 added closed-loop speed and lateral control in simulation and a tuned hardware controller with calibrated encoder feedback and steering range. Milestone 4 introduced path planning capabilities across structured simulation tracks (Empty, Racing, and City) and integrated physical sensors (encoders and IMU) for full state measurement via Arduino-to-Pi communication. Finally, Milestone 5 culminated in full system integration. It incorporated Kalman-filtered localization and closed the autonomy loop, enabling both simulated and physical vehicles to autonomously navigate lanes, avoid obstacles, and handle curvatures.

Index Terms—ROS 2, Jazzy, Ackermann steering, Gazebo Harmonic, teleoperation, closed-loop control, Arduino, autonomous vehicle, Kalman filter, path planning

I. INTRODUCTION

Autonomous wheeled vehicles require a working chain from computation and middleware to vehicle motion before higher-level perception, localization, and planning algorithms can be evaluated. For an Ackermann-steered platform, this chain must preserve both the software communication model and the physical constraints imposed by front-wheel steering geometry.

This paper presents the cumulative documentation for Team 23 in the MCTR1002 Autonomous Systems course at the German University in Cairo (GUC). Milestone 1 validated the ROS 2 Jazzy environment on hardware and simulation platforms. Milestone 2 then introduced the first complete driving modes in simulation and the first actuator-level hardware tests. Milestone 3 added closed-loop control for speed and lateral tracking, along with tuned hardware drive and steering behavior. Milestone 4 integrated obstacle-avoidance path planning in simulation and full hardware sensor odometry. Milestone 5 realized complete system integration featuring Kalman filter state estimation and autonomous track navigation. The project is therefore documented as a staged development process rather than as isolated submissions.

The contributions covered in this paper are:

- 1) ROS 2 Jazzy validation on Raspberry Pi 4 hardware and a Gazebo Harmonic simulation laptop;
- 2) an Ackermann vehicle model launched in an empty Gazebo world and connected to ROS 2 through `ros_gz_bridge`;
- 3) OLR and teleoperation ROS 2 Python nodes for commanding and logging vehicle motion;
- 4) closed-loop speed and lateral ROS 2 controllers with tunable gains;
- 5) a tuned Arduino actuator controller that parses serial commands, drives the DC motor and steering servo, and applies feedforward plus proportional correction using encoder feedback;
- 6) a planning node to generate velocity profiles and desired lanes to avoid obstacles across multiple Gazebo track

environments;

- 7) integration of IMU and encoder feedback for hardware state measurement sent via serial to a Raspberry Pi;
- 8) a complete localization module employing a Kalman filter to estimate vehicle states from noisy sensor data, successfully used to complete physical autonomous driving challenges.

Section II reviews the technical context. Section III describes the overall architecture. Sections IV, V, and VI document the Milestone 1 foundation, Milestone 2 simulation work, and Milestone 2 hardware work. Section VII covers the Milestone 3 closed-loop simulation and tuned hardware controller. Section VIII details the path planning and sensor integration of Milestone 4. Section IX presents the final Milestone 5 system integration and track results. Section X summarizes the achieved deliverables and Section XI outlines the next development steps.

II. RELATED WORK

A. Robotics Middleware

ROS 2 uses a decentralized Data Distribution Service (DDS) communication model that supports modular robotics systems, where perception, planning, control, and hardware interfaces can be developed as separate nodes [1], [2]. This publish-subscribe structure matches the project's staged development: first validating communication, then adding driving nodes, and later integrating autonomy modules.

B. Ackermann Steering Geometry

The classic Ackermann constraint aligns all wheels with a common instantaneous center of rotation. The geometric relation is

$$\cot \delta_o - \cot \delta_i = \frac{T}{L}, \quad (1)$$

where δ_o and δ_i are the outer and inner steering angles, T is the track width, and L is the wheelbase [3]. The simulation model used in Milestone 2 relies on a Gazebo Ackermann steering system with explicit wheelbase, wheel separation, wheel radius, steering-joint, and wheel-joint parameters.

C. Localization, Planning, and Control

Future milestones require state estimation, path planning, and closed-loop motion control. EKF-based fusion of encoder and inertial measurements is widely used for Ackermann mobile robots [4], [5]. Frenet-frame planning and MPC-based tracking are common choices for autonomous-driving trajectories [6], [7]. Electronic differential and actuator-level control approaches further motivate early hardware testing of steering and drive systems [8].

III. SYSTEM ARCHITECTURE

The project is split into simulation and hardware paths that share the same command intent: a desired longitudinal speed and a desired steering angle. The simulation path sends those commands to Gazebo as a *Twist* message, while the

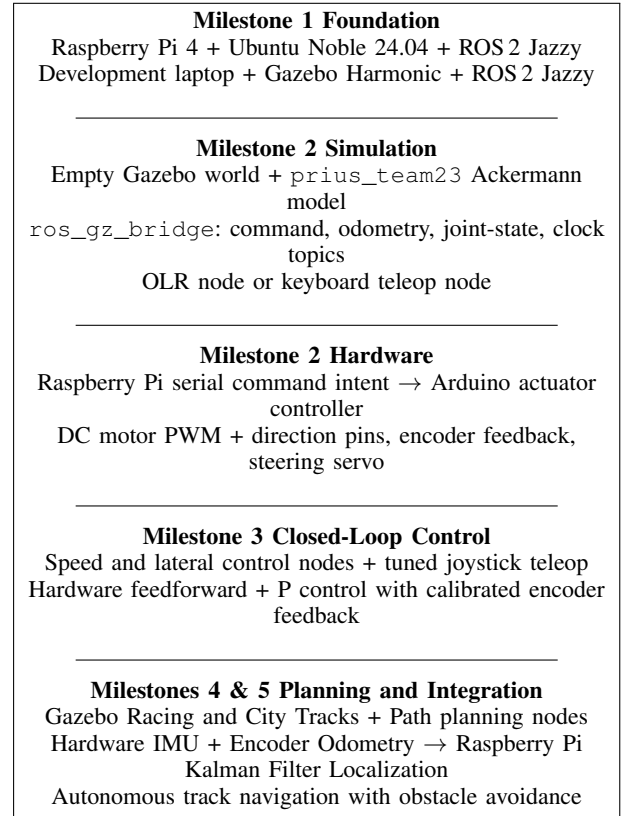


Fig. 1. Cumulative system architecture through Milestone 5.

hardware path sends an ASCII serial command to an Arduino controller.

The ROS2 package remains an `ament_python` package named `Autonomous_Systems_Project_Team_23`. Milestone 2 adds launch and model assets, the OLR driving node, and the teleoperation driving node while retaining the validation and publisher-subscriber foundation from Milestone 1. Milestone 3 adds closed-loop speed and lateral controllers, an MS3 launch file, and a joystick teleoperation node aligned with the hardware serial protocol.

IV. MILESTONE 1: ROS 2 ENVIRONMENT VALIDATION

A. Hardware Platform

The hardware platform is a Raspberry Pi4 with 4GB RAM running Ubuntu Noble 24.04 on ARM64. ROS 2 Jazzy was installed from the official APT repository and sourced through `/opt/ros/jazzy/setup.bash`. The `ros2 doctor` tool reported all five health checks passing, covering network, RMW, package, Python, and middleware configuration checks.

A custom validation node printed a confirmation message at 1 Hz on the Raspberry Pi. Fig. 3 shows continuous terminal output over approximately 26 s.

B. Simulation Platform

The simulation platform is an ASUS ROG GA401QE laptop with an AMD Ryzen 9 5900HX CPU and NVIDIA GTX 1650

TABLE I
MILESTONE 1 HARDWARE DIAGNOSTIC RESULTS

Diagnostic item	Status
Network interface	PASS
RMW implementation	PASS
ROS 2 package versions	PASS
Python environment	PASS
Middleware configuration	PASS
Overall	5/5 PASS

The screenshot shows the output of the `ros2 doctor` command on a Raspberry Pi 4. It lists various system components and their status, all of which are 'PASS'. The components include network interface, RMW implementation, ROS 2 package versions, Python environment, and middleware configuration. The overall status is '5/5 PASS'.

Fig. 2. Raspberry Pi 4 ROS2 Jazzy diagnostics with all `ros2 doctor` checks passing.

GPU. ROS 2 Jazzy and Gazebo Harmonic were installed with the `ros-jazzy-ros-gz` bridge package. A validation node confirmed ROS 2 execution on the simulation machine before vehicle-level integration.

The screenshot shows the output of the `ros2 run` command for the `validation_printing_node` on a Raspberry Pi. It displays a series of log messages indicating that the node is running successfully on Team 23 Raspberry Pi. The messages include timestamps and node names, confirming the successful execution of the validation node.

Fig. 3. Milestone 1 hardware validation node running on the target Raspberry Pi platform.

The screenshot shows the output of the `ros2 daemon start` command in a terminal window. It displays the output of the `friction-less@GA401QE:~$ ros2 run Autonomous_Systems_Project_Team_23 validation_node` command, showing a log message from the `validation_printing_node` on Team 23 Raspberry Pi.

Fig. 4. Milestone 1 simulation-side ROS 2 validation output.

V. MILESTONE 2: SIMULATION DRIVING MODES

A. Gazebo Vehicle Launch

Milestone 2 requires an Ackermann-like vehicle to be launched in an empty Gazebo world and driven through ROS 2 [9]. The project package includes a `prius_team23` model with an SDF description, mesh assets, and a Gazebo Ackermann steering plugin. The plugin binds the front steering joints and four wheel joints, with a steering limit of 0.5 rad, wheelbase of 1.0 m, wheel separation of 1.25 m, and wheel radius of 0.3 m.

The Milestone 2 launch file starts Gazebo Harmonic using `empty.sdf`, spawns the vehicle model at the requested lane offset, starts the ROS-Gazebo bridge, and launches either the OLR node or the teleoperation node. It can also start `rqt_graph` for graph inspection.

TABLE II
MILESTONE 2 ROS-GAZEBO BRIDGE INTERFACES

Interface	ROS 2 type	Role
Clock bridge	Clock	Simulation time
Command bridge	Twist	Vehicle command
Odometry bridge	Odometry	Vehicle state
Joint-state bridge	JointState	Steering feedback

B. Open-Loop Response Node

The OLR node reads launch parameters for the command topic, state topic, joint-state topic, publish rate, wheelbase, desired speed, desired steering, maximum speed, and maximum turn rate. It clamps the requested speed, converts steering-angle intent into yaw rate using

$$\dot{\psi} = \frac{v \tan \delta}{L}, \quad (2)$$

and publishes the resulting `Twist` command at 10Hz. The node also subscribes to odometry and joint states, then logs steering angle, velocity, position, and yaw.

The screenshot shows the Gazebo Harmonic simulation environment. A blue car model is visible on a grid floor. The top panel displays a list of ROS 2 topics and their data, including `autonomous_system_m_2_ol_team_23/state` and `autonomous_system_m_2_ol_team_23/steering`. The bottom panel shows the Entity Tree with a list of entities including `default`, `ground_plane`, `sun`, and `vehicle`.

Fig. 5. Milestone 2 OLR demonstration in Gazebo Harmonic. The ROS 2 node publishes a constant command and logs odometry and steering feedback.

C. Teleoperation Node

The teleoperation node provides an interactive driving mode using keyboard arrow keys. The up and down arrows increase or decrease longitudinal speed, while the left and right arrows adjust steering. The space key sends a stop command and the Q key requests shutdown. Steering is rate-limited before conversion to yaw rate to avoid instantaneous jumps in the simulated command.

The same node includes optional serial forwarding for later embedded use. When enabled, it sends speed and steering commands to an Arduino in the form `SPD:<speed>, STR:<steering>`. This keeps the simulation teleoperation interface aligned with the hardware actuator command interface.

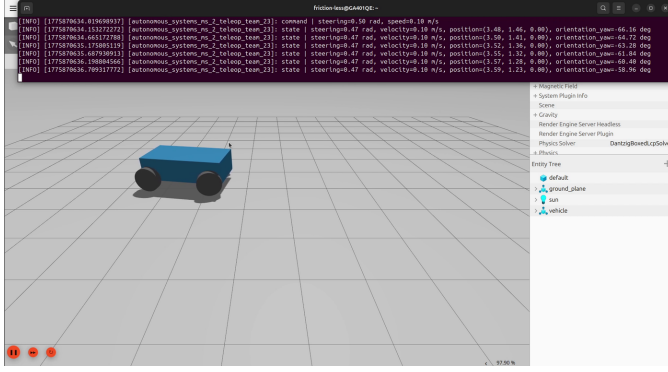


Fig. 6. Milestone 2 teleoperation mode. Keyboard commands update the requested speed and steering while odometry and joint-state feedback are logged on screen.

VI. MILESTONE 2: HARDWARE ACTUATION

The hardware deliverable tested the physical vehicle's drive and steering actuators. The submitted Arduino sketch configures an H-bridge motor driver, steering servo, and quadrature encoder inputs. It receives serial commands from the host, parses target speed and steering values, and applies a 20 Hz control loop with PID speed regulation.

A. Serial Command Interface

The serial protocol uses ASCII messages:

`SPD:<speed>, STR:<steering>`

where speed is expressed in m/s and steering is expressed in radians. A 500 ms command timeout forces the target speed to zero and clears the integral term. This protects the vehicle from continuing to drive if command streaming stops.

B. Motor and Steering Control

The controller estimates wheel speed from encoder ticks using the configured pulses per revolution and wheel diameter. It computes the speed error, applies proportional, integral, and derivative terms, constrains the output to the available PWM range, and selects motor direction based on the sign of the target speed. Steering commands are normalized by the maximum steering angle and mapped to a bounded servo angle around a 90-degree center position.



Fig. 7. Milestone 2 physical actuator test on the assembled scaled vehicle. The Arduino controller drives the DC motor and steering servo during outdoor testing.

VII. MILESTONE 3: CLOSED-LOOP CONTROL AND TUNED HARDWARE

Milestone 3 advanced the platform from open-loop driving to closed-loop speed and lateral control in simulation, and updated the hardware controller with measured calibration data. A closed-loop speed node publishes a filtered speed command on an internal topic and limits acceleration to reduce command spikes. The lateral controller subscribes to odometry and the speed command, applies a lateral PID plus heading feedback, and publishes the final `Twist` command to Gazebo. Both controllers share parameters for wheelbase, speed limits, and steering limits to keep the software behavior aligned with the vehicle model.

The MS3 launch file spawns the vehicle with explicit initial pose arguments and exposes gain parameters for speed and lateral tuning. It starts the ROS–Gazebo bridge, then launches the speed and lateral controllers after the model is spawned to ensure topic availability.

On hardware, the Arduino controller was retuned with measured encoder counts and tested motor response. The control loop runs at 50 Hz, applies feedforward scaling based on a calibrated maximum speed, and adds a proportional correction to reject load disturbances. Steering commands are mapped to a widened servo range, and a 2 s command timeout halts the vehicle if serial streaming stops. These changes improve both steady-state speed tracking and steering authority relative to the earlier milestone.

VIII. MILESTONE 4: PATH PLANNING AND HARDWARE SENSORY INTEGRATION

Building upon the low-level closed-loop controllers established in previous milestones, Milestone 4 introduced high-

level decision making in simulation and full state measurement on hardware.

A. Simulation Track Environments and Path Planning

To evaluate the autonomous capabilities of the vehicle, three distinct Gazebo worlds were constructed: an Empty 2-lane Track, a 2-lane Racing Track featuring discrete obstacle blocks, and a curved City Track. The vehicle model was spawned in these environments using updated launch files.

A planning ROS 2 node was implemented to generate safe trajectory references. Given knowledge of the track layout and predefined obstacle coordinates, the planner computes a desired velocity profile and a target lane. It publishes the longitudinal velocity commands to the speed controller and desired lateral offsets to the lateral (Stanley) controller. For the Racing Track, the planner safely triggers lane changes in advance of static obstacles to maintain collision-free forward progress before returning to the original lane.

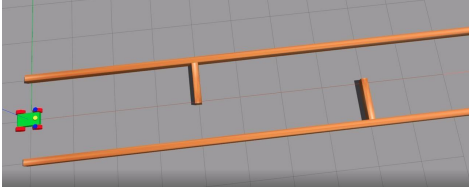


Fig. 8. Path planning execution in the Gazebo Racing Track world, demonstrating lane changing to avoid obstacles.

B. Hardware State Estimation

On the physical platform, accurate control relies on precise state feedback. The vehicle was equipped with quadrature encoders and an Inertial Measurement Unit (IMU) to measure speed (v_d) and heading (θ) respectively.

The Arduino firmware was upgraded to continuously poll these sensors and compute a discrete-time wheel odometry approximation:

$$v_x = v_d \cos(\theta) \quad (3)$$

$$v_y = v_d \sin(\theta) \quad (4)$$

$$x_{k+1} = x_k + v_x \Delta t \quad (5)$$

$$y_{k+1} = y_k + v_y \Delta t \quad (6)$$

This state vector $[v_d, x, y, \theta]$ is transmitted at high frequency over the serial link to the Raspberry Pi, where a Localization ROS 2 node receives and logs the vehicle's pose, closing the gap between physical motion and the software stack.

IX. MILESTONE 5: SYSTEM INTEGRATION AND LOCALIZATION

Milestone 5 represents the culmination of the project architecture, merging perception (localization), planning, and control into a unified, headless autonomous execution.

A. Kalman Filter State Estimation

Real-world sensors suffer from high-frequency noise. To maximize state estimation reliability, a discrete linear Kalman Filter was implemented within the localization stack node running on the Raspberry Pi [4]. The vehicle state vector is defined as $x_k = [x, y, \theta, v]^T$. The system propagates the state estimate forward in time during the prediction step via:

$$\hat{x}_{k|k-1} = A\hat{x}_{k-1|k-1} + Bu_k \quad (7)$$

$$P_{k|k-1} = AP_{k-1|k-1}A^T + Q \quad (8)$$

where A represents the state transition matrix, B is the control input matrix, P is the system state covariance estimate, and Q is the calibrated process noise covariance matrix representing system uncertainties [5].

When a transmission telemetry packet is parsed from the serial buffer, the localization node executes a correction update to reconcile the internal model with the physical encoder and IMU observations (z_k):

$$K_k = P_{k|k-1}H^T (HP_{k|k-1}H^T + R)^{-1} \quad (9)$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (z_k - H\hat{x}_{k|k-1}) \quad (10)$$

$$P_{k|k} = (I - K_kH) P_{k|k-1} \quad (11)$$

where K_k denotes the calculated optimal Kalman gain, H represents the measurement mapping matrix, and R is the sensor measurement noise covariance matrix populated using standard deviations derived from individual sensor datasheets.

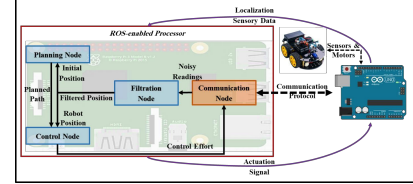


Fig. 9. Block diagram showing the connections between the hardware components and the ROS 2 nodes running on the Raspberry Pi.

B. Hardware System Integration and Track Testing

The filtered state topics were routed directly to the longitudinal and lateral controllers, which were now running locally on the Raspberry Pi alongside the planner. The system operated headlessly via SSH.

The physical vehicle was tested across progressively difficult scenarios:

- 1) **1-Lane Longitudinal Motion:** The vehicle accurately tracked a desired speed profile while maintaining zero lateral error.
- 2) **2-Lane Obstacle Avoidance:** The vehicle autonomously identified its position, followed a planned lane-change maneuver to avoid a physical obstacle, and returned to its original lane.
- 3) **City Track Curvatures:** The vehicle successfully navigated continuous turns, relying on IMU heading feedback and encoder speeds to maintain lane centering.

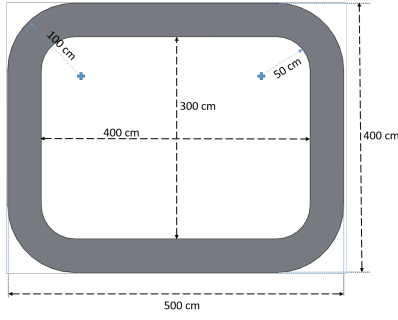


Fig. 10. The physical scaled vehicle navigating the test track using onboard ROS 2 nodes and Kalman-filtered odometry.

These successful tests validated the complete ROS 2 architecture, proving that the localized computations on the Arduino and the high-level ROS nodes on the Raspberry Pi can effectively collaborate to achieve autonomous navigation.

X. RESULTS

Table III summarizes the cumulative project status through Milestone 5.

TABLE III
CUMULATIVE DELIVERABLE STATUS

Milestone	Deliverable group	Status
1	Hardware ROS 2 diagnostics and validation node	✓
1	Simulation ROS 2/Gazebo validation and pub-sub node	✓
2	Empty-world Ackermann model launch and bridge	✓
2	OLR and keyboard teleoperation driving modes	✓
2	Arduino motor, encoder, PID, and steering actuator test	✓
3	Closed-loop speed and lateral control in simulation	✓
3	Tuned hardware controller with calibrated encoder feedback	✓
4	Gazebo track worlds and path planning nodes	✓
4	Hardware IMU/encoder integration and odometry	✓
5	Kalman filter localization implementation	✓
5	Full hardware autonomous track navigation	✓

The project now features a complete, repeatable path for both simulated and physical autonomous vehicle motion. The architectural separation between low-level actuation (Arduino) and high-level decision making (Raspberry Pi) proved robust across all tested scenarios.

XI. LIMITATIONS AND FUTURE WORK

While Milestone 5 successfully demonstrated autonomous obstacle avoidance and lane following based on predetermined map knowledge and odometry, the vehicle lacks active environmental perception. The following items remain for future development:

- activate perception sensors such as LiDAR and camera streams for dynamic obstacle detection and visual lane tracking;
- transition from predefined trajectory rules to dynamic Frenet-frame planning and Model Predictive Control (MPC);
- implement Simultaneous Localization and Mapping (SLAM) to navigate unstructured, unknown environments;

- refine the Kalman filter into an Extended Kalman Filter (EKF) or Unscented Kalman Filter (UKF) to better handle the non-linearities of Ackermann steering kinematics at higher speeds.

XII. CONCLUSION

This paper presented the cumulative development of Team 23's autonomous Ackermann vehicle through Milestones 1–5. Milestone 1 validated ROS 2 Jazzy on the hardware and simulation platforms. Milestone 2 extended that foundation by launching the vehicle in Gazebo, adding teleoperation, and testing physical actuators. Milestone 3 introduced closed-loop control in simulation and tuned hardware feedback. Milestone 4 implemented structured simulation tracks with path planning algorithms and integrated physical IMU and encoder sensors for complete state measurement. Finally, Milestone 5 realized full system integration. By utilizing a Kalman filter for robust localization, the vehicle successfully executed autonomous obstacle avoidance and lane-following maneuvers headlessly on the physical track. These deliverables establish a fully functional, end-to-end autonomous architecture ready for advanced perception and dynamic environment mapping.

REFERENCES

- [1] S. Jain *et al.*, "Blockchain and autonomous vehicles: Recent advances and future directions," *IEEE Access*, vol. 9, pp. 130 264–130 328, 2021.
- [2] Open Robotics, "ROS 2 Jazzy Jalisco Documentation," <https://docs.ros.org/en/jazzy/>, 2026, accessed: 2026-04-25.
- [3] T. Xu *et al.*, "Application of Ackermann steering in obstacle crossing platform of six-wheeled robots," in *Proc. 2nd Int. Symposium on Control Engineering and Robotics (IS CER)*, 2023.
- [4] M. Peiris *et al.*, "Kalman filter-based sensor fusion for Ackermann steering mobile robots," *Journal of Physics: Conference Series*, vol. 2811, no. 1, p. 012004, 2024.
- [5] I. Georgiev, "Vehicle localization for autonomous driving by sensor fusion," *IEEE Transactions on Intelligent Vehicles*, 2025, DOI not verified from available metadata at staging time.
- [6] M. Wang *et al.*, "A real-time dynamic trajectory planning for autonomous driving vehicles," in *Proc. 3rd Conf. on Vehicle Control and Intelligence (CVCI)*, 2019.
- [7] H. Wu *et al.*, "MPC-based obstacle avoidance path tracking control for distributed drive electric vehicles," *World Electric Vehicle Journal*, vol. 13, no. 12, 2022.
- [8] S. Zhang *et al.*, "Electronic differential control for four-Ackermann-steering-wheeled electric vehicles," *IEEE Transactions on Vehicular Technology*, 2022.
- [9] Open Robotics, "Gazebo Harmonic Documentation," <https://gazebo.sim.org/docs/harmonic/>, 2026, accessed: 2026-04-25.